StageCraft

UE5 C++ 기반 데이터 주도 Stage / Step 콘텐츠 시스템

개발 목적

상용 프로젝트에서 기존 Guide / Stage 시스템을 활용하고 확장한 경험을 바탕으로, 개인 프로젝트에서는 요구사항을 다시 정의하고 작은 규모의 콘텐츠 실행 구조를 직접 설계·구현했습니다.

핵심 키워드

UE5 C++
Data-Driven Content
Stage / Step
Action / Condition
Runtime State · Delegate

프로젝트 개요

Engine	Unreal Engine 5.4.4 / C++ / Visual Studio 2022
형태	소규모 3D 콘텐츠 시스템 포트폴리오
핵심 목표	Action / Condition 조합으로 Stage 콘텐츠를 구성하는 구조
현재 상태	Stage 1 · Stage 2 및 Level 전환 구현

구현 방향

- Stage는 Step 배열로 구성하고, 각 Step은 Objective / Actions / Conditions를 가집니다.
- StageInstance는 Action / Condition 타입을 직접 분기하지 않고 공통 기반 클래스로 실행과 평가를 진행합니다.
- CameraShake · Delay · PlaySequence를 추가하고 기존 Action과 조합해 Stage 2를 구성했습니다.

PLAY DEMO

StageCraft — Stage 1 → Stage 2 연속 플레이 (약 1분 19초)

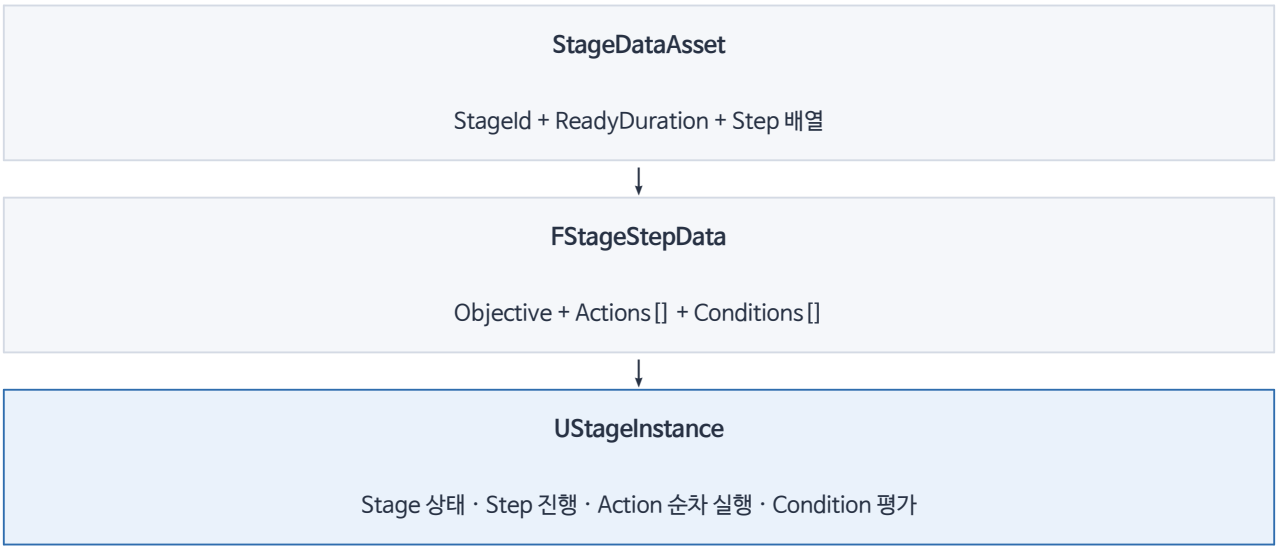
https://youtu.be/UnL3wGZvGz8

포트폴리오에서 보여주고 싶은 것

콘텐츠 요구사항을 데이터, 실행 흐름, 개별 동작과 판정으로 나누고 실제 플레이에 연결하는 과정을 보여주는 것을 목표로 합니다.

Stage → Step → Action / Condition

콘텐츠 데이터와 진행 Runtime의 역할을 나누고, StageInstance가 공통 실행 흐름을 담당합니다.



Action	Condition
동작 실행 Execute → OnCompleted	진행 조건 평가 Satisfied / NotSatisfied / Invalid

Objective – 현재 Step의 목표 텍스트와 월드 마커 정보를 담으며, Step 시작 시 변경 이벤트를 통해 UI / 월드 마커에 전달합니다.

1. Step 시작	Objective 변경을 알리고 Action 배열 실행
2. Action 실행	배열 순서대로 Execute()를 호출하고 완료 결과를 Delegate로 전달
3. Condition 평가	모든 Action 완료 후 Condition을 AND 방식으로 평가하며, 미충족 시 다음 Tick에 다시 평가
4. 진행 / 실패	모든 Condition 충족 시 다음 Step으로 진행하며, Invalid 또는 Action 실패 시 Stage를 실패 처리

Subsystem 책임과 수명

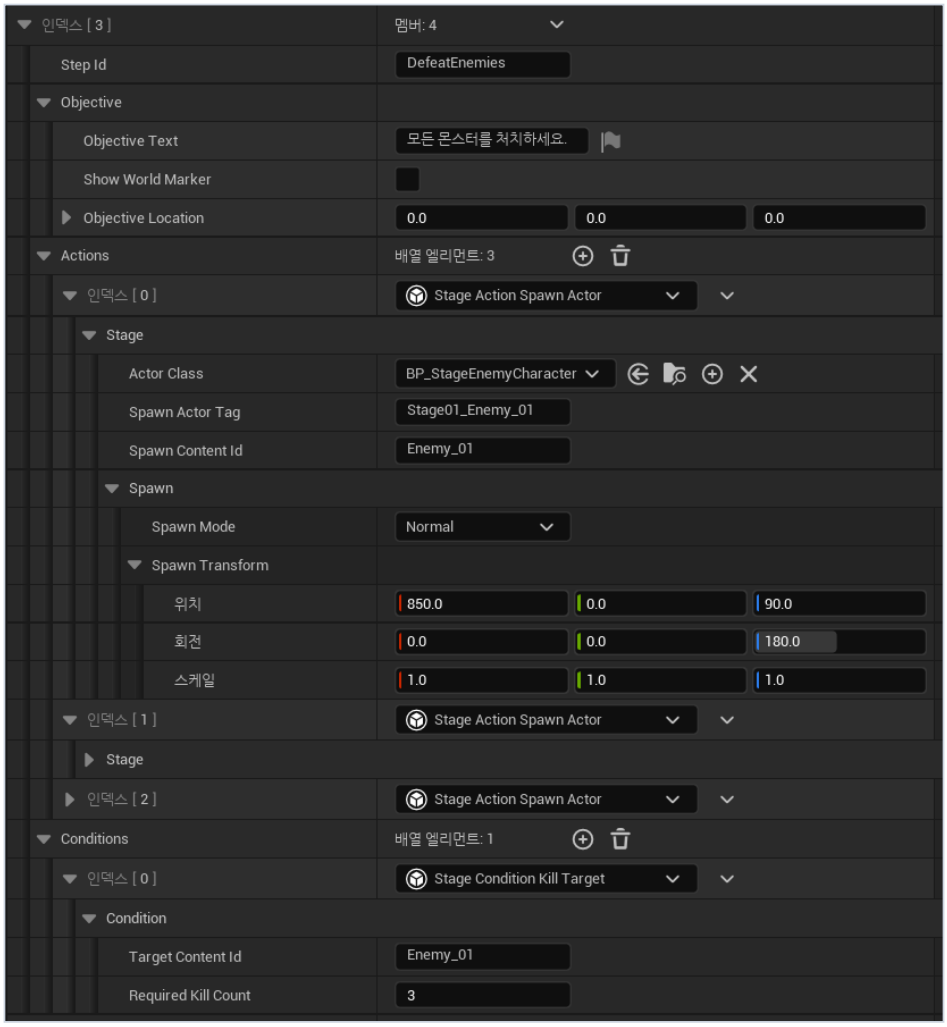
StageSubsystem	UWorldSubsystem · 현재 World의 StageInstance 생성 및 관리
StageFlowSubsystem	UGameInstanceSubsystem · Stage 선택 및 Level 전환 관리

설계 포인트

StageInstance는 SpawnActor, ShowDialog, KillTarget 같은 개별 구현 타입을 직접 분기하지 않습니다. 공통 기반 클래스를 통해 실행 흐름을 관리하고, 실제 동작과 판정은 각 파생 클래스가 담당합니다.

Stage 1 — DefeatEnemies

SpawnActor로 Enemy를 생성하고 KillTarget으로 처치 조건을 평가합니다.



Action / Condition 구성

SpawnActor

SpawnMode를 Normal로 설정해 DataAsset에 지정한 위치에 Enemy 3명을 생성합니다. 생성된 Enemy에는 Stage 진행에 사용할 ContentId를 설정합니다.

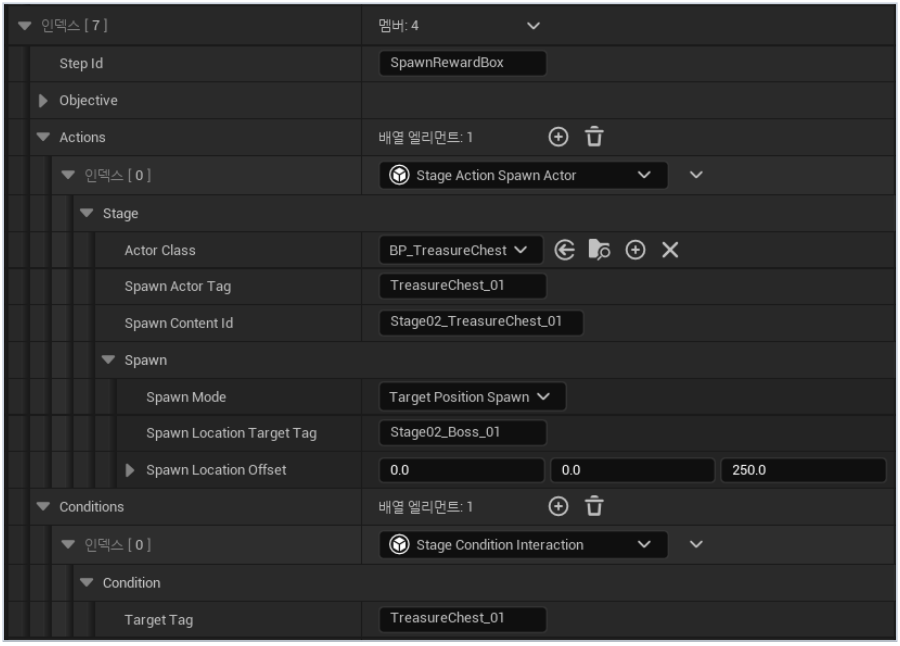
Enemy Death / KillTarget

Enemy 사망 시 ContentId를 StageInstance에 전달하고, StageInstance는 해당 ContentId의 처치 수를 누적합니다. KillTarget Condition은 RequiredKillCount(3) 충족 여부를 평가하여 다음 Step 진행을 결정합니다.

Stage 2 — Action 재사용과 보스 연출

동일한 SpawnActor를 보물상자 생성에 사용하고, PlaySequence를 연출 Action으로 설정했습니다.

SpawnRewardBox | SpawnActor + Interaction



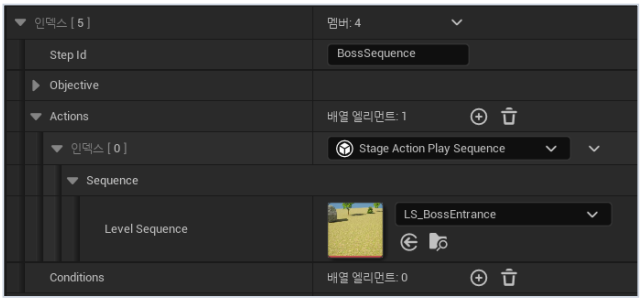
SpawnActor 재사용

Stage 1에서는 SpawnMode를 Normal로 설정해 지정 위치에 Enemy를 생성했습니다. Stage 2에서는 동일한 SpawnActor에 TargetPositionSpawn을 설정하고, SpawnLocationTargetTag로 지정한 보스 위치에 Z축 250 Offset을 적용해 보물상자를 생성합니다.

Interaction Condition

보물상자의 Tag를 조건에 연결하고, 플레이어가 상호작용하면 다음 Step으로 진행하도록 구성했습니다.

BossSequence | PlaySequence



PlaySequence Action에 LS_BossEntrance를 설정하여 보스 등장 연출을 재생합니다. Sequence가 자연 종료되면 OnFinished 이벤트를 통해 Action 완료를 알리고 다음 Action으로 진행합니다.

대표 소스 파일 — 총 14개

데이터 정의, Action / Condition의 공통 인터페이스, StageInstance의 진행 제어 로직,
주요 Action / Condition 구현 및 Stage 전환 구조를 확인할 수 있도록 선정했습니다.

1. Stage 데이터 정의

StageDataAsset.h / StageStepData.h

Stage와 Step의 데이터 구조를 정의합니다.
각 Step에 Objective, Action, Condition을 조합하여 콘텐츠 진행 흐름을 구성하는 방식을 확인할 수 있습니다.

2. Action / Condition 공통 계약

StageAction.h / StageCondition.h

Action 실행 및 완료 통지, Condition 평가를 위한 공통 계약을 정의합니다.
StageInstance가 개별 Action / Condition 구현 클래스에 직접 의존하지 않고 실행과 평가를 처리할 수 있는 기반입니다.

3. Stage 실행 관리

StageInstance.h / StageInstance.cpp

Stage와 Step의 진행 상태를 관리하고, Action을 순차적으로 실행하며 Condition을 평가합니다.
공통 기반 클래스를 통해 다양한 Action / Condition을 처리하는 핵심 Runtime입니다.

4. Actor 생성 Action

StageAction_SpawnActor.h / StageAction_SpawnActor.cpp

설정된 Actor를 생성하고, 생성 위치와 콘텐츠 식별 정보를 적용합니다.
동일한 Action을 여러 Stage에서 서로 다른 데이터 설정으로 재사용하는 사례입니다.

5. 처치 조건 평가

StageCondition_KillTarget.h / StageCondition_KillTarget.cpp

Stage Runtime에 기록된 대상별 처치 정보를 조회하여 목표 처치 수 충족 여부를 평가합니다.
공통 Condition을 상속해 처치 조건을 구현한 사례입니다.

대표 소스 파일 — 총 14개 (계속)

Level Sequence 연동, Stage 간 전환과 StageInstance의 주요 함수들을 확인할 수 있습니다.

6. Level Sequence 연출 Action

StageAction_PlaySequence.h / StageAction_PlaySequence.cpp

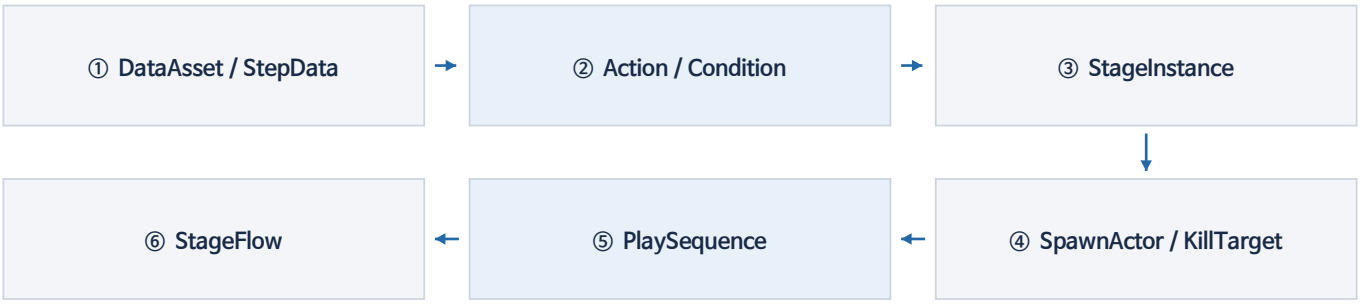
Unreal Engine의 Level Sequence를 재생하고, 자연 종료 시 엔진의 완료 이벤트를 공통 Action 완료 통지로 연결합니다.
기존 StageInstance의 실행 로직을 변경하지 않고 새로운 연출 기능을 Action으로 확장했습니다.

7. Stage 간 Level 전환

StageFlowSubsystem.h / StageFlowSubsystem.cpp

UGameInstanceSubsystem을 활용하여 현재 Stage 식별, 다음 Stage 선택 및 Level 전환을 관리합니다.
World에 종속된 Stage 실행과 Level 전환 이후에도 유지되어야 하는 진행 정보를 분리한 설계 사례입니다.

권장 읽기 순서



StageInstance 주요 함수와 역할

StartStep()	Step 데이터 확인 및 Objective 변경
ExecuteCurrentAction()	Action을 순서대로 실행하고 완료 결과 처리
TryCompleteStep()	Action 완료 후 Condition 평가 시작
CheckConditions()	Satisfied / NotSatisfied / Invalid 결과 확인
AdvanceStep() / Fail()	다음 Step 진행 또는 Stage 실패 처리

Stage 1 → Stage 2 연속 플레이

서로 다른 Level과 StageDataAsset으로 구성한 두 Stage의 실제 진행 흐름입니다.

Stage 1 — VillageDefense

01	**NPC 조사**	Interaction Condition
02	**의뢰 대화**	ShowDialog / DialogComplete
03	**목표 지역 이동**	ReachLocation
04	**일반 적 3명 처치**	SpawnActor / KillTarget
05	**보상 및 Stage 완료**	Reward

Stage 간 Level 전환

Stage 1 완료 → Next Stage → StageFlowSubsystem / OpenLevel → Stage 2 시작

Stage 2 — BossEncounter

01	**NPC 접근 및 연출**	ReachLocation / CameraShake / Delay
02	**NPC 대화**	Interaction / ShowDialog
03	**보스 등장 연출**	ReachLocation / PlaySequence
04	**보스 처치**	KillTarget
05	**보물상자 상호작용**	SpawnActor / Interaction
06	**보상 및 Stage 완료**	Delay / Reward

향후 개선 계획

1. 비동기 Action 안정성

Stage 종료 시 완료 이벤트 확인과 실행 중 Action의 취소·정리 처리 보완

2. Runtime 상태 관리

DataAsset의 Action / Condition 설정과 실행별 상태의 분리 검토

3. Debug / Data Validation

Stage · Step · Condition 상태 확인 및 잘못된 데이터·참조 설정 검사